

KMeans_Online_Retail_Analytics

October 24, 2024

0.1 K-Means Clustering

Overview Online retail is a transnational data set which contains all the transactions occurring between 01/12/2010 and 09/12/2011 for a UK-based and registered non-store online retail. The company mainly sells unique all-occasion gifts. Many customers of the company are wholesalers.

The steps are broadly: 1. Read and understand the data 2. Clean the data 3. Prepare the data for modelling 4. Modelling 5. Final analysis and reco

1 1. Read and visualise the data

```
[4]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import datetime as dt

import sklearn
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

from scipy.cluster.hierarchy import linkage
from scipy.cluster.hierarchy import dendrogram
from scipy.cluster.hierarchy import cut_tree
```

```
[5]: # read the dataset
retail_df = pd.read_csv("Online_Retail.csv", sep=",", encoding="ISO-8859-1",
    ↳header=0)
retail_df.head()
```

```
-----
FileNotFoundError                                Traceback (most recent call last)
Cell In[5], line 2
      1 # read the dataset
----> 2 retail_df = pd.read_csv("Online_Retail.csv", sep=",",
    ↳encoding="ISO-8859-1", header=0)
```

```
3 retail_df.head()
```

```
File ~/.local/lib/python3.11/site-packages/pandas/util/_decorators.py:211, in
↳ deprecate_kwarg.<locals>._deprecate_kwarg.<locals>.wrapper(*args, **kwargs)
    209     else:
    210         kwargs[new_arg_name] = new_arg_value
--> 211 return func(*args, **kwargs)
```

```
File ~/.local/lib/python3.11/site-packages/pandas/util/_decorators.py:331, in
↳ deprecate_nonkeyword_arguments.<locals>.decorate.<locals>.wrapper(*args,
↳ **kwargs)
    325 if len(args) > num_allow_args:
    326     warnings.warn(
    327         msg.format(arguments=_format_argument_list(allow_args)),
    328         FutureWarning,
    329         stacklevel=find_stack_level(),
    330     )
--> 331 return func(*args, **kwargs)
```

```
File ~/.local/lib/python3.11/site-packages/pandas/io/parsers/readers.py:950, in
↳ read_csv(filepath_or_buffer, sep, delimiter, header, names, index_col,
↳ usecols, squeeze, prefix, mangle_dupe_cols, dtype, engine, converters,
↳ true_values, false_values, skipinitialspace, skiprows, skipfooter, nrows,
↳ na_values, keep_default_na, na_filter, verbose, skip_blank_lines, parse_dates,
↳ infer_datetime_format, keep_date_col, date_parser, dayfirst, cache_dates,
↳ iterator, chunksize, compression, thousands, decimal, lineterminator,
↳ quotechar, quoting, doublequote, escapechar, comment, encoding,
↳ encoding_errors, dialect, error_bad_lines, warn_bad_lines, on_bad_lines,
↳ delim_whitespace, low_memory, memory_map, float_precision, storage_options)
    935 kwds_defaults = _refine_defaults_read(
    936     dialect,
    937     delimiter,
    (... )
    946     defaults={"delimiter": ","},
    947 )
    948 kwds.update(kwds_defaults)
--> 950 return _read(filepath_or_buffer, kwds)
```

```
File ~/.local/lib/python3.11/site-packages/pandas/io/parsers/readers.py:605, in
↳ _read(filepath_or_buffer, kwds)
    602 _validate_names(kwds.get("names", None))
    604 # Create the parser.
--> 605 parser = TextFileReader(filepath_or_buffer, **kwds)
    607 if chunksize or iterator:
    608     return parser
```

```
File ~/.local/lib/python3.11/site-packages/pandas/io/parsers/readers.py:1442, in
↳ TextFileReader.__init__(self, f, engine, **kwds)
    1439     self.options["has_index_names"] = kwds["has_index_names"]
    1441 self.handles: IOHandles | None = None
```

```
-> 1442 self._engine = self._make_engine(f, self.engine)
```

File ~/.local/lib/python3.11/site-packages/pandas/io/parsers/readers.py:1735, in

```
↳ TextFileReader._make_engine(self, f, engine)
    1733     if "b" not in mode:
    1734         mode += "b"
-> 1735 self.handles = get_handle(
    1736     f,
    1737     mode,
    1738     encoding=self.options.get("encoding", None),
    1739     compression=self.options.get("compression", None),
    1740     memory_map=self.options.get("memory_map", False),
    1741     is_text=is_text,
    1742     errors=self.options.get("encoding_errors", "strict"),
    1743     storage_options=self.options.get("storage_options", None),
    1744 )
    1745 assert self.handles is not None
    1746 f = self.handles.handle
```

File ~/.local/lib/python3.11/site-packages/pandas/io/common.py:856, in

```
↳ get_handle(path_or_buf, mode, encoding, compression, memory_map, is_text,
↳ errors, storage_options)
    851 elif isinstance(handle, str):
    852     # Check whether the filename is to be opened in binary mode.
    853     # Binary mode does not support 'encoding' and 'newline'.
    854     if ioargs.encoding and "b" not in ioargs.mode:
    855         # Encoding
--> 856         handle = open(
    857             handle,
    858             ioargs.mode,
    859             encoding=ioargs.encoding,
    860             errors=errors,
    861             newline="",
    862         )
    863     else:
    864         # Binary mode
    865         handle = open(handle, ioargs.mode)
```

FileNotFoundError: [Errno 2] No such file or directory: 'Online_Retail.csv'

```
[ ]: # basics of the df
retail_df.info()
```

2 2. Clean the data

```
[ ]: # missing values
round(100*(retail_df.isnull().sum())/len(retail_df), 2)

[ ]: # drop all rows having missing values
retail_df = retail_df.dropna()
retail_df.shape

[ ]: retail_df.head()

[ ]: # new column: amount
retail_df['amount'] = retail_df['Quantity']*retail_df['UnitPrice']
retail_df.head()
```

3 3. Prepare the data for modelling

- R (Recency): Number of days since last purchase
- F (Frequency): Number of transactions
- M (Monetary): Total amount of transactions (revenue contributed)

```
[ ]: # monetary
grouped_df = retail_df.groupby('CustomerID')['amount'].sum()
grouped_df = grouped_df.reset_index()
grouped_df.head()

[ ]: # frequency
frequency = retail_df.groupby('CustomerID')['InvoiceNo'].count()
frequency = frequency.reset_index()
frequency.columns = ['CustomerID', 'frequency']
frequency.head()

[ ]: # merge the two dfs
grouped_df = pd.merge(grouped_df, frequency, on='CustomerID', how='inner')
grouped_df.head()

[ ]: retail_df.head()

[ ]: # recency
# convert to datetime
retail_df['InvoiceDate'] = pd.to_datetime(retail_df['InvoiceDate'],
                                          format='%d-%m-%Y %H:%M')

[ ]: retail_df.head()
```

```
[ ]: # compute the max date
max_date = max(retail_df['InvoiceDate'])
max_date

[ ]: # compute the diff
retail_df['diff'] = max_date - retail_df['InvoiceDate']
retail_df.head()

[ ]: # recency
last_purchase = retail_df.groupby('CustomerID')['diff'].min()
last_purchase = last_purchase.reset_index()
last_purchase.head()

[ ]: # merge
grouped_df = pd.merge(grouped_df, last_purchase, on='CustomerID', how='inner')
grouped_df.columns = ['CustomerID', 'amount', 'frequency', 'recency']
grouped_df.head()

[ ]: # number of days only
grouped_df['recency'] = grouped_df['recency'].dt.days
grouped_df.head()

[ ]: # 1. outlier treatment
plt.boxplot(grouped_df['recency'])

[ ]: # two types of outliers:
# - statistical
# - domain specific

[ ]: # removing (statistical) outliers
Q1 = grouped_df.amount.quantile(0.05)
Q3 = grouped_df.amount.quantile(0.95)
IQR = Q3 - Q1
grouped_df = grouped_df[(grouped_df.amount >= Q1 - 1.5*IQR) & (grouped_df.
    ↳ amount <= Q3 + 1.5*IQR)]

# outlier treatment for recency
Q1 = grouped_df.recency.quantile(0.05)
Q3 = grouped_df.recency.quantile(0.95)
IQR = Q3 - Q1
grouped_df = grouped_df[(grouped_df.recency >= Q1 - 1.5*IQR) & (grouped_df.
    ↳ recency <= Q3 + 1.5*IQR)]

# outlier treatment for frequency
Q1 = grouped_df.frequency.quantile(0.05)
Q3 = grouped_df.frequency.quantile(0.95)
IQR = Q3 - Q1
```

```
grouped_df = grouped_df[(grouped_df.frequency >= Q1 - 1.5*IQR) & (grouped_df.  
↪frequency <= Q3 + 1.5*IQR)]
```

```
[ ]: # 2. rescaling  
rfm_df = grouped_df[['amount', 'frequency', 'recency']]  
  
# instantiate  
scaler = StandardScaler()  
  
# fit_transform  
rfm_df_scaled = scaler.fit_transform(rfm_df)  
rfm_df_scaled.shape
```

```
[ ]: rfm_df_scaled = pd.DataFrame(rfm_df_scaled)  
rfm_df_scaled.columns = ['amount', 'frequency', 'recency']  
rfm_df_scaled.head()
```

4 4. Modelling

```
[ ]: # k-means with some arbitrary k  
kmeans = KMeans(n_clusters=4, max_iter=50)  
kmeans.fit(rfm_df_scaled)
```

```
[ ]: kmeans.labels_
```

```
[ ]: # help(KMeans)
```

4.1 Finding the Optimal Number of Clusters

4.1.1 SSD

```
[ ]: # elbow-curve/SSD  
ssd = []  
range_n_clusters = [2, 3, 4, 5, 6, 7, 8]  
for num_clusters in range_n_clusters:  
    kmeans = KMeans(n_clusters=num_clusters, max_iter=50)  
    kmeans.fit(rfm_df_scaled)  
  
    ssd.append(kmeans.inertia_)  
  
# plot the SSDs for each n_clusters  
# ssd  
plt.plot(ssd)
```

4.1.2 Silhouette Analysis

$$\text{silhouette score} = \frac{p - q}{\max(p, q)}$$

p is the mean distance to the points in the nearest cluster that the data point is not a part of

q is the mean intra-cluster distance to all the points in its own cluster.

- The value of the silhouette score range lies between -1 to 1.
- A score closer to 1 indicates that the data point is very similar to other data points in the cluster,
- A score closer to -1 indicates that the data point is not similar to the data points in its cluster.

```
[ ]: # silhouette analysis
range_n_clusters = [2, 3, 4, 5, 6, 7, 8]

for num_clusters in range_n_clusters:

    # initialise kmeans
    kmeans = KMeans(n_clusters=num_clusters, max_iter=50)
    kmeans.fit(rfm_df_scaled)

    cluster_labels = kmeans.labels_

    # silhouette score
    silhouette_avg = silhouette_score(rfm_df_scaled, cluster_labels)
    print("For n_clusters={0}, the silhouette score is {1}".
          format(num_clusters, silhouette_avg))
```

```
[ ]: # final model with k=3
kmeans = KMeans(n_clusters=3, max_iter=50)
kmeans.fit(rfm_df_scaled)
```

```
[ ]: kmeans.labels_
```

```
[ ]: # assign the label
grouped_df['cluster_id'] = kmeans.labels_
grouped_df.head()
```

```
[ ]: # plot
sns.boxplot(x='cluster_id', y='amount', data=grouped_df)
```

4.2 Hierarchical Clustering

```
[ ]: rfm_df_scaled.head()
```

```
[ ]: grouped_df.head()
```

```
[ ]: # single linkage
mergings = linkage(rfm_df_scaled, method="single", metric='euclidean')
dendrogram(mergings)
plt.show()
```

```
[ ]: # complete linkage
mergings = linkage(rfm_df_scaled, method="complete", metric='euclidean')
dendrogram(mergings)
plt.show()
```

```
[ ]: # 3 clusters
cluster_labels = cut_tree(mergings, n_clusters=3).reshape(-1, )
cluster_labels
```

```
[ ]: # assign cluster labels
grouped_df['cluster_labels'] = cluster_labels
grouped_df.head()
```

```
[ ]: # plots
sns.boxplot(x='cluster_labels', y='recency', data=grouped_df)
```

```
[ ]: # plots
sns.boxplot(x='cluster_labels', y='frequency', data=grouped_df)
```

```
[ ]: # plots
sns.boxplot(x='cluster_labels', y='amount', data=grouped_df)
```